Lab 2

- Due Jan 17 by 10p.m.
- Points 1

Note: The autotesting is not quite ready for this lab. It will be updated on Tuesday.

Lab 2: Pointers and Input

Introduction

The purpose of this lab is to practice writing C programs involving pointers, arrays, and the `scanf` function.

To start, log into MarkUs and click on the lab. You can use the same process as Lab 1 to access the starter files: click the button at the bottom to add the files to your repository, then do a `git pull` to download the files to your computer.

TIP: Remember that you must not commit to your repository any executable files. You can make sure of this by using `git add` explicitly on the files you mean to add. For this lab, you could run `git add invest.c phone.c score_card.c digits.c`, and then commit and push.

We've described each problem briefly below, but for more detail on the first two problems, read through the starter code. There is no starter code for the final two problems.

1. invest.c

Your task is to implement a function `invest` that takes an amount of money and multiplies it by a given rate. It's your job to figure out the exact type of this function's arguments, given the sample usage in the `main` function in the starter code.

2. score_card.c

Your task is to implement a function `sum_card`, which takes an array of pointers to integers, and returns the sum of the integers being pointed to.

3. phone.c

Your task is to write a small C program called `phone.c` that uses `scanf` to read two values from standard input. The first is a 10 character string and the second is an integer. The program takes no command-line arguments. (There is no starter code for this question. You will write this program from scratch, so remember to add, commit, and push.)

If the integer is -1, the program prints the full string to `stdout`. If the integer i is between 0 and 9, the program prints only the corresponding character (i.e., at index i) from the string to `stdout`. In both of these cases the program returns 0.

If the integer is less than -1 or greater than 9, the program prints the message "ERROR" to `stdout` and returns 1.

We haven't learned about strings yet, but you will see that to hold a string of 10 characters, you actually need to allocate space for 11 characters. The extra space is for a special character that indicates the end of the string. Use this line

```
char phone[11];
```

to declare the variable to hold the string. Other than this line, there is no starter code for this program.

Note: the program must not print any prompts.

Here is some sample runs of the program ($ is the shell prompt). The line after `./phone` is input typed by the user:

```
$ ./phone
4161234567 3
1
$ ./phone
4161234567 -1
4161234567
$ ./phone
4161234567 9
7
$ ./phone
4161234567 10
ERROR
```

4. digits.c

Complete the two functions in `digits.c` according to their description in the comments. Your functions should not make assumptions about the length of the array of digits passed in.

Note that we could have written the function signature in two equivalent ways:

```
int repeated_digits(int *numbers, int length) {
int repeated_digits(int numbers[], int length) {
```

In both cases we can treat `numbers` as an array in the body of the function, and we still need to pass in the `length` of the array.

Submission

Use git to submit your final `invest.c`, `score_card.c`, `phone.c` and `digits.c` -- make sure they're inside your `lab2` folder and named exactly as described in this handout, as that's where our test scripts will be looking for them. Do *NOT* add or commit executables to your repository. We will build executables by compiling your code as part of testing it.

Log into MarkUs to check that the files are in the right place.

IMPORTANT: make sure to review how to use git to submit your work to the MarkUs server; in particular, you need to run `git push`, not just `git add` and `git commit`.

Running a Simple Test on MarkUs

For each of the four programs, we have provided a simple test that will confirm that you have submitted a file that compiles without warnings and runs one easy test case. As a last step run these tests using the Run Tests button in the Automated Testing tab. If your submission does not pass these automated tests, it will get 0, but if it passes these tests, it it not necessarily fully correct. You still need to test that your code works correctly in all cases.